

Programming for Essential Digital Skills, Chapter 7

Christoph Walsh

2026

Numpy

- The built-in Python functions are not well suited to performing many mathematical operations we often need to do.
- The operations can be done, but often require several lines of code or complicated list comprehensions.
- The numpy module contains many functions that can easily do these common operations.
- The conventional way to load numpy is with:

```
import numpy as np
```

- We will do some examples to show how useful numpy is.

Doubling All Numbers in a List

```
x = [2, 4, 8]
y = []                # For loop approach
for i in x:
    y.append(2 * i)
y
```

[4, 8, 16]

```
[2 * i for i in x]  # List comprehension approach
```

[4, 8, 16]

```
2 * np.array(x)      # Numpy approach
```

```
array([ 4,  8, 16])
```

Element-By-Element Multiplication

Suppose we have two lists of equal length:

$$x = (x_1, x_2, \dots, x_n)$$

$$y = (y_1, y_2, \dots, y_n)$$

and we want to calculate z from this which is:

$$z = (x_1 \times y_1, x_2 \times y_2, \dots, x_n \times y_n)$$

Element-By-Element Multiplication

```
x = [2, 4, 8]
y = [3, 2, 2]
z = []                                # Iterating over the indices of x and
for i in range(len(x)):
    z.append(x[i] * y[i])
z
```

[6, 8, 16]

```
z = []
for i, j in zip(x, y):                # Iterating over x and y with zip
    z.append(i * j)
z
```

[6, 8, 16]

Element-By-Element Multiplication

```
# List comprehension approach:  
[i * j for i, j in zip(x, y)]
```

[6, 8, 16]

```
# Numpy approach:  
x = np.array(x)  
y = np.array(y)  
x * y
```

array([6, 8, 16])

The Median of a List of Numbers

- When you sort a list of n numbers, the median is:
 - ▶ The number in the middle when n is odd
 - ▶ The average of the two middle numbers when n is even

```
def median(x):  
    y = sorted(x)  
    if len(y) % 2 == 0:  
        return (y[len(y) // 2 - 1] + y[len(y) // 2]) / 2  
    else:  
        return y[len(y) // 2]  
median([2, 6, 4])
```

4

```
median([2, 6, 4, 3])
```

3.5

The Median of a List of Numbers

In numpy, we can do this much more easily:

```
x = np.array([2, 6, 4])  
np.median(x)
```

```
np.float64(4.0)
```

```
x = [2, 6, 4, 3]  
np.median(x)
```

```
np.float64(3.5)
```

Matrix Multiplication

- Suppose we want to calculate $C = AB$, where:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \\ 3 & 1 & 3 \end{pmatrix} \quad B = \begin{pmatrix} 2 & 1 & 2 \\ 3 & 2 & 1 \\ 1 & 3 & 1 \end{pmatrix}$$

- Row i and column j of C is given by:

$$c_{ij} = \sum_{k=1}^3 a_{ik} b_{kj}$$

For example:

$$c_{21} = \sum_{k=1}^3 a_{2k} b_{k1} = a_{21} b_{11} + a_{22} b_{21} + a_{23} b_{31} = 2 \times 2 + 3 \times 3 + 1 \times 1 = 14$$

Defining Matrices with Lists:

- We can define matrices as nested lists.
- Each element of the list is a list which represents a row of the matrix.

```
A = [  
    [1, 2, 3],  
    [2, 3, 1],  
    [3, 1, 3]  
]
```

```
B = [  
    [2, 1, 2],  
    [3, 2, 1],  
    [1, 3, 1]  
]
```

Matrix Multiplication without Numpy

```
C = [  
    [0, 0, 0],  
    [0, 0, 0],  
    [0, 0, 0]  
]  
for i in range(3):  
    for j in range(3):  
        C[i][j] += A[i][0] * B[0][j] + \  
                    A[i][1] * B[1][j] + \  
                    A[i][2] * B[2][j]  
for row in C:  
    print(row)
```

```
[11, 14, 7]  
[14, 11, 8]  
[12, 14, 10]
```

Matrix Multiplication without Numpy (Improved)

```
C = [  
    [0, 0, 0],  
    [0, 0, 0],  
    [0, 0, 0]  
]  
for i in range(3):  
    for j in range(3):  
        for k in range(3):  
            C[i][j] += A[i][k] * B[k][j]  
for row in C:  
    print(row)
```

```
[11, 14, 7]  
[14, 11, 8]  
[12, 14, 10]
```

Matrix Multiplication with Numpy

```
import numpy as np
A = np.array(A)
B = np.array(B)
np.dot(A, B)
```

```
array([[11, 14,  7],
       [14, 11,  8],
       [12, 14, 10]])
```